

MEDIATEK

MT8676 Android IPO User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-08-12	刘锋	正式版

目录

版本记录	2
目录	3
图片目录	4
表格目录	4
1 IPO 概述	5
1.1 基本介绍	5
1.2 为什么需要 IPO	5
2 名词解释	6
3 架构概述	7
3.1 Android IPO 架构	7
3.2 Android 系统中硬件电源管理的两种情形	7
3.3 各硬件对应进出 IPO 的电源管理方案	8
3.3.1 不同模式下 CPU 和进程的状态	9
3.3.2 IPO 接口说明	10
3.3.3 IPO 接口调用时序	10
4 配置/客制化指南	13
4.1 HU 端配套修改	13
4.2 代码改动介绍	13
4.3 USB 连接	14
4.4 如何客制化进入和退出 IPO 过程动画	14
4.5 自定义 IPO 白名单	14
4.6 在退出 IPO 时，杀死进程	15
4.7 进入/退出 IPO 时，通知 Java 进程	16
4.8 进入/退出 IPO 时，如何通知 Native 进程	16
5 常见问题解答	17
5.1 关闭 IPO 时产生的关键日志	17
5.2 用命令开启 IPO 时产生的关键日志	17
5.3 用电源键开启 IPO 时产生的关键日志	18
5.4 因碰撞开启 IPO 时产生的关键日志	18
5.5 进入 IPO 后，如何排查功耗高的原因	18

5.6 IPO 调试命令说明.....19

附件一 附加条款.....20

图片目录

图 3-1. IPO 架构.....7

图 3-2. IPO 关机流程.....11

图 3-3. 退出 IPO 开机流程.....12

图 4-1. USB 模式切换.....14

图 4-2. 动画客制化.....14

图 4-3. Hard code 方式设置白名单.....15

图 4-4. 白名单示例代码.....15

图 4-5. 通知 Native 进程的示例代码.....16

表格目录

表 2-1. 名词解释.....6

表 3-1. 各硬件进出 IPO 的电源管理方案.....8

表 3-2. 不同模式下 CPU 和进程的状态.....10

表 3-3. IPO 接口说明.....10

表 4-1. IPO 相关代码修改.....13

1 IPO 概述

1.1 基本介绍

本文档主要介绍了 MT8676 IPO 快速开机功能并提供了 IPO 常见问题的分析方法。

IPO，全称 Instant Power On，即快速开机，对于汽车 ACC On 之后快速进入倒车画面以及启动车载录像功能非常实用。

1.2 为什么需要 IPO

- 在当前智能汽车控制系统中，设备启动时间是衡量性能的重要指标。
- IPO 是一种软件解决方案，设备可以在可配置的时间内启动，且无需额外的硬件成本，它利用 Android Suspend 功能创造了一种新的用户体验。
- 在设备进入 IPO 期间，系统中运行的大多数进程和服务将被 ActivityManagerService 停止或杀死，以防止意外行为；当系统进入 Suspend 状态之后，用户可以通过外部事件或远程事件唤醒系统以退出 IPO 模式，从而实现快速启动。
- 用户可能每天都会点火、熄火，对开关速度有要求，不想等待过久。
- 即使用户感觉车机已经关机，但这只是一种假关机，实际上没有真的关机，而是进入 early_suspend 或者 Suspend 状态，车机仍可以接受远程指令。

2 名词解释

表 2-1. 名词解释

缩略词	全称及释义
HU	Head Unit 主机
IHUU	Infotainment Head Unit 信息娱乐主机
IPO	Instant Power On 快速开机
IVI	In-Vehicle Infotainment 车载信息娱乐系统
T-Box	Telematics-BOX 车联网控制单元

3 架构概述

3.1 Android IPO 架构

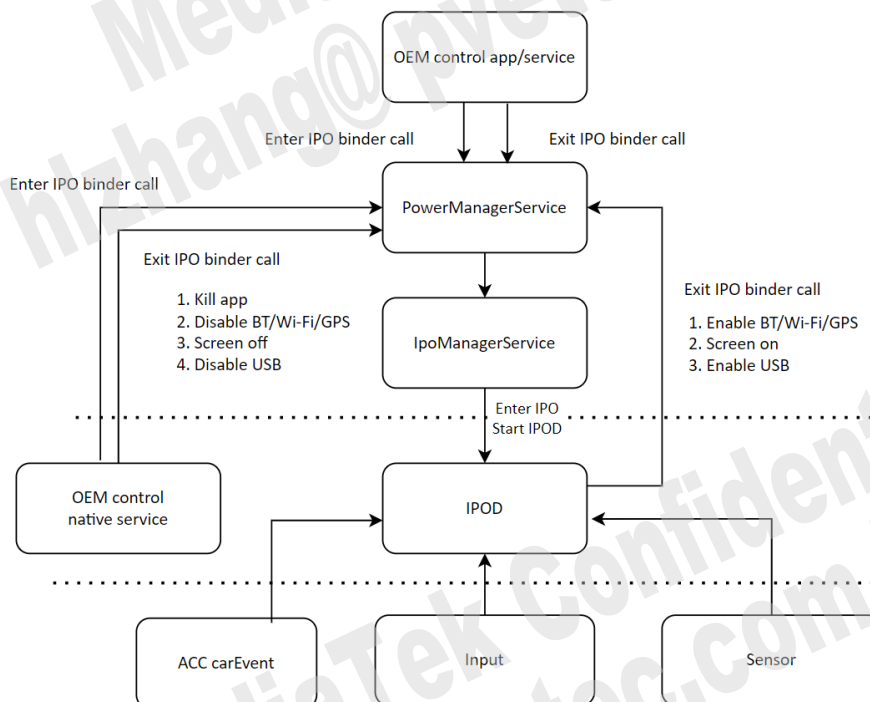


图3-1. IPO 架构

3.2 Android 系统中硬件电源管理的两种情形

1. 独立的电源管理逻辑，不依赖 Kernel Suspend 流程（比如 Camera/Display 等）。驱动程序发现没有人访问设备时就掉电，有人访问时就上电并获取 WakeLock。

2. 依赖 Kernel Suspend/Resume 架构

- HW Driver（硬件驱动）实现 Kernel Driver（内核驱动）要求的 Suspend/Resume 函数。
- 若没有人持有 WakeLock，系统进入 Suspend 状态时会调用每个 HW Driver 的 Suspend()。
- 系统被唤醒，进入 Resume 状态时也会调用每个 HW Driver 的 Resume()。
- HW Driver 独立实现 Suspend/Resume 函数来自行控制上电/掉电。

以上两种情形对应的电源管理方案如下：

1. IPO 关机时，触发所有使用者放弃访问硬件设备：第 1 阶段掉电。

2. IPO 关机时，清除硬件使用场景，释放 WakeLock，进入 Suspend 状态：第 2 阶段掉电。

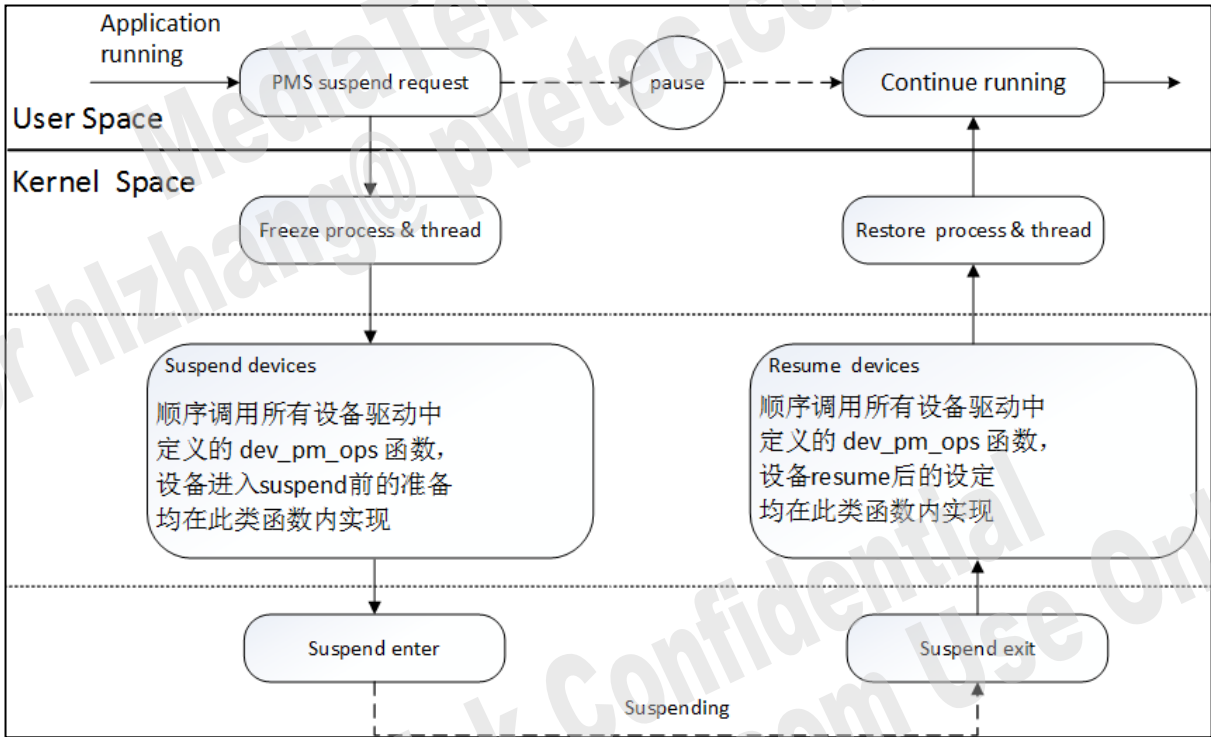


Figure 3-1. Suspend/Resume 流程

3.3 各硬件对应进出 IPO 的电源管理方案

表 3-1. 各硬件进出 IPO 的电源管理方案

模块	Suspend 阶段	进入 IPO 关机处理	退出 IPO 开机处理
GPS	2	IPO 杀 HU 端 AP 来清除 GPS 应用场景 停止使用 GPS; GPS SW 进 Suspend	IPO 开机不做处理
BT	1	IPO 会关 BT; BT SW 进 Suspend	IPO 会开 BT
WIFI	1	IPO 会关 WIFI; WIFI SW 进 Suspend	IPO 会开 WIFI
TTY	2	清除 TTY 使用场景 Kernel Suspend -> TTY Driver Suspend	IPO 开机不做处理 Kernel Resume -> TTY Driver Resume
SPI	2	清除 SPI 使用场景 Kernel Suspend -> SPI Driver Suspend	IPO 开机不做处理 Kernel Resume -> 对应 Driver Resume

模块	Suspend 阶段	进入 IPO 关机处理	退出 IPO 开机处理
Camera	1	IPO 杀 HU 端 AP 来清除摄像头应用场景 EVS 收到 ACC OFF 会停止摄像头使用 摄像头驱动会上电	IPO 开机不做处理 若有人使用摄像头，摄像头驱动会上电
Sensor	2	IPO 杀 HU 端 AP 来清除传感器应用场景 是否还有传感器使用场景 Kernel Suspend -> Sensor Driver Suspend	IPO 开机不做处理 Kernel Resume -> 对应 Driver Resume
EPROM	2	IPO 杀 HU 端 AP 来关闭 EPROM 应用场景 是否还有其他使用场景 Kernel Suspend -> Sensor Driver Suspend	IPO 开机不做处理 Kernel Resume -> 对应 Driver Resume
Display	1	IPO 调用 PMS_FW 进 Early Suspend Display Driver 会给 Display 断电	1. IPO 开机会调用 PMS_FW 开背光 2. Display Driver 通过 GPIO 通知 MCU
Mic noise elimination	2	进 IPO 关机,配合移除使用场景 驱动程序进 Suspend 后掉电	IPO 开机不做处理 Kernel Resume -> 对应 Driver Resume
Mic D/A conversion	2	进 IPO 关机,配合移除使用场景 驱动程序进 Suspend 后掉电	IPO 开机不做处理 Kernel Resume -> 对应 Driver Resume
USB Hub	2	进 IPO 关机,配合移除使用场景 驱动程序进 Suspend 后掉电	IPO 开机不做处理 Kernel Resume -> 对应 Driver Resume
Ethernet	2	进 IPO 关机,配合移除使用场景 驱动程序进 Suspend 后掉电	IPO 开机不做处理 Kernel Resume -> 对应 Driver Resume

3.3.1 不同模式下 CPU 和进程的状态

- 软件进程状态：Running (占用 CPU 运行)，Suspend (主动放弃 CPU 资源等待执行)，Killed (杀死)
- CPU 状态：Wakeup (亮屏)，Early Suspend (唤醒状态 & 灭屏)，Suspend (休眠)

表 3-2. 不同模式下 CPU 和进程的状态

	MT8676 Android			
	CPU	T-Box 进程	Java Core 进程/Native 进程	任务进程
全工作模式	Early Suspend/ Suspend/Wakeup	Suspend/Running IPO 假关机状态	Suspend/Running	Suspend/Running
半工作模式	Early Suspend	Running	Running	Killed
休眠模式	Suspend	Suspend	Suspend	Killed

3.3.2 IPO 接口说明

表 3-3. IPO 接口说明

接口/结构	描述
public static void lowpowerStartIPO()	PowerManagerService 提供的 IPO 关机接口 1. 杀白名单之外的 App，关闭 BT/WIFI/GPS 等外设，熄屏。 2. AP 释放 WakeLock 后，系统进入休眠模式。
public static void lowpowerStopIPO(int flag)	PowerManagerService 退出 IPO 开机接口 参数：0 (熄屏)，1 (亮屏) 系统进入全工作模式。

3.3.3 IPO 接口调用时序

3.3.3.1 进入 IPO 关机流程

- 红色方框：白名单之外的 App 可以接收 ACTION_SHUTDOWN_HU 广播并做清理动作。
- 紫色方框：调用 Android 层接口，关外设，移除外设使用场景。
- 黄色方框：杀死非白名单内 App 和黑名单内 Native 进程。

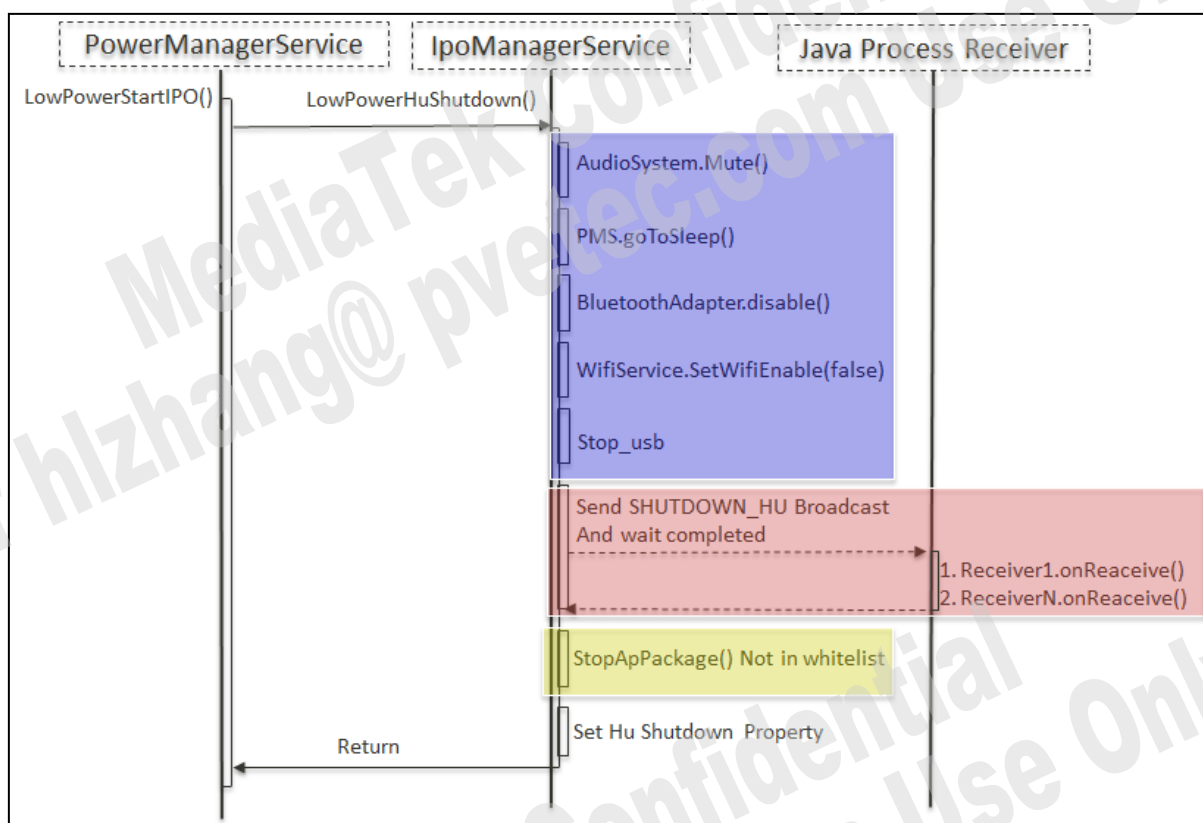


图3-2. IPO 关机流程

3.3.3.2 退出 IPO 开机流程

点火唤醒中断/Modem 电话短信 -> 唤醒 Kernel -> 执行退出 IPO

开机动画和开机流程并行处理。

- 紫色方框：在发广播前主动唤醒在第一阶段关闭的设备。
- 红色方框：可以接收 ACTION_BOOT_HU 广播做初始化动作。
- 黄色方框：发出 Android 原生开机广播，启动期望开机自启动的 App。

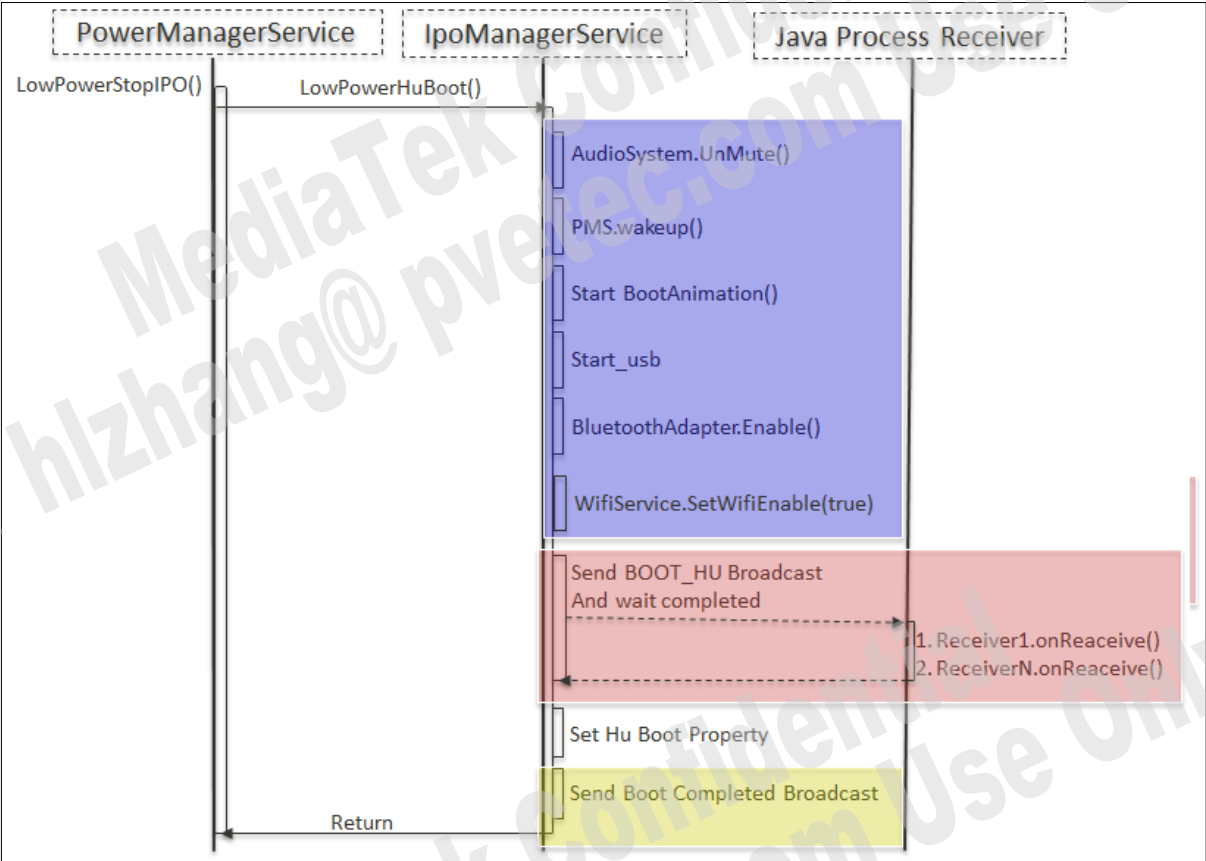


图3-3. 退出IPO 开机流程

4 配置/客制化指南

4.1 HU 端配套修改

客制化的应用程序可以监听以下 2 个广播:

- com.mediatek.intent.action.ACTION_SHUTDOWN_HU: 关机前做一些处理
- com.mediatek.intent.action.ACTION_BOOT_HU: 开机前做一些处理

4.2 代码改动介绍

表 4-1. IPO 相关代码修改

AOSP Repo	修改文件	修改原因
frameworks/base	core/java/android/os/IPowerManager.aidl	新增进入 IPO Service 接口
frameworks/base	core/res/AndroidManifest.xml	定义 IPO 开机关机广播
frameworks/base	services/core/java/com/android/server/power/PowerManagerService.java	新增进入 IPO 服务接口 IPO 关机, 跳过 WakeUp 请求
frameworks/base	services/core/Android.bp	加载 MediaTek IPO Jar
frameworks/base	services/core/java/com/android/server/policy/PhoneWindowManager.java	IPO 关机, 跳过 key 事件
vendor/mediatek/proprietary/frameworks/opt/ipod	java/com/mediatek/ipomanager/IpoManagerService.java	IPO 核心类
vendor/mediatek/proprietary/external/ipod	*	进入 IPO 后, IPOD 检测到 power key 事件后退出 IPO

4.3 USB 连接

进入 IPO 后，断开 USB 需要切换到 Device 模式，才能休眠；退出 IPO 时，再切换到 Host 模式，防止 USB Kernel 持锁导致无法进 Suspend；退出 IPO 后，恢复 USB 连接。

代码路径：vendor/mediatek/proprietary/external/ipod/ipod.rc

```
18 on property:sys.ipo.disable.usb=1
19   write /sys/class/udc/musb-hdrc/device/cmode 0
20
21 on property:sys.ipo.disable.usb=0
22   write /sys/class/udc/musb-hdrc/device/cmode 1
```

图 4-1. USB 模式切换

4.4 如何客制化进入和退出 IPO 过程动画

代码路径：vendor/mediatek/proprietary/frameworks/opt/ipo/IpoManagerService.java

```
304 private IpoManagerService(Context context) {
305     myLog( "start IpoManagerService");
306     if(SystemProperties.get("persist.sys.ipo.debug", "").equals("1")){
307         localLOGV = true;
308     }
309     mWelcomPackage = SystemProperties.get("ro.odm.welcomeactivity.package", "com.incalls.welcomescreen");
310     mWelcomClass = SystemProperties.get("ro.odm.welcomeactivity.class", "com.incalls.activity.WelcomeActivity");
311     mBootAnimService = SystemProperties.get("ro.odm.bootanimation.name", "bootanim");
312     mShutAnimService = SystemProperties.get("ro.odm.shutdownanimation.name", "bootanim");
313     mBootAnimTime = Integer.valueOf(SystemProperties.get("ro.odm.bootanimation.time", "2"));
314     mShutAnimTime = Integer.valueOf(SystemProperties.get("ro.odm.shutdownanimation.time", "2"));
315     MAX_BOOT_TIME = mBootAnimTime*1000+1000;
316     MAX_SHUTDOWN_TIME = mShutAnimTime*1000+1000;
```

图 4-2. 动画客制化

默认 bootanim 时间是 2 秒，这个时间可以根据需求客制化。

4.5 自定义 IPO 白名单

白名单里的进程，在进入 IPO 时不会被杀死。

代码路径：vendor/mediatek/proprietary/frameworks/opt/ipo/IpoManagerService.java

- 使用 Hard code 方式设置白名单，请参考下图：

```

215     private static final String[] mHardCodeShutdownList = {
216         "system",
217         "com.android.bluetooth",
218         "com.android.phone",
219         "android.process.acore",
220         "com.android.systemui",
221         "com.android.car",
222         "com.android.car.carlauncher",
223         "com.android.providers.media.module",
224     };

```

图4-3. Hard code 方式设置白名单

- 使用 Dynamic 方式设置白名单：

setprop persist.ipo.shutdown.process.wl

如/com.xx.yy/com.zz.ww，请参考下图：

```

245     // Customization +
246     String processList = SystemProperties.get("persist.ipo.shutdown.process.wl", null);
247     if ( processList != null ) {
248         myLog("Process whitelist: " + processList);
249         ArrayList<String> processArrayList = new ArrayList<String>();
250         parseStringIntoArray("/", processList, processArrayList);
251
252         for (int i = 0; i < processArrayList.size(); ++i) {
253             String item = processArrayList.get(i);
254             if (item.length() > 0) {
255                 if (item.startsWith("!") && item.length() > 1
256                     && sShutdownWhiteList.contains(item.substring(1))) {
257                     sShutdownWhiteList.remove(item.substring(1));
258                 } else if (!item.startsWith("!") && !item.matches("^\\s*$")
259                     && !sShutdownWhiteList.contains(item)) {
260                     sShutdownWhiteList.add(item);
261                 }
262             }
263         }
264     }
265     // Customization -

```

图4-4. 白名单示例代码

4.6 在退出 IPO 时，杀死进程

配置 persist.ipo.prebootkill.list。

4.7 进入/退出 IPO 时，通知 Java 进程

当 IPO 关闭的时候，接收 `com.mediatek.intent.action.ACTION_SHUTDOWN_HU` 广播。

当 IPO 开启的时候，接收 `com.medaitek.intent.action.ACTION_BOOT_HU` 广播。

4.8 进入/退出 IPO 时，如何通知 Native 进程

将属性触发器流程添加到 `init.rc` 文件。

示例代码：

```
12 on property:sys.hu.status=shutdown-done
13 | start vendor.ipod
14
15 on property:sys.hu.status=boot-done
16 | stop vendor.ipod
```

图 4-5. 通知 Native 进程的示例代码

5 常见问题解答

5.1 关闭 IPO 时产生的关键日志

关键词：LowPowerStartIPO/ IpoManagerService/ IPODMAIN/ipod

```
03-07 08:30:57.955 1188 1320 D PowerManagerService: LowPowerStartIPO //PMS IPO
entrance
03-07 08:30:57.956 1188 1320 D CarTbox : [IpoManagerService] Power_status_2.2
//IpoManagerService entrance
03-07 08:30:57.956 1188 1320 D CarTbox : [IpoManagerService] lowPowerHuShutdown start
03-07 08:30:57.957 1188 1401 D CarTbox : [IpoManagerService]handleMessage msg.what =
SHUT_ANIMATION_START
03-07 08:30:57.957 1188 1401 D CarTbox : [IpoManagerService] startShutAnimation
03-07 08:30:57.966 1188 1401 D CarTbox : [IpoManagerService]handleMessage msg.what = AUDIO_STOP
03-07 08:30:57.966 1188 1401 D CarTbox : [IpoManagerService]muteSystem
03-07 08:30:57.967 1188 1401 D CarTbox : [IpoManagerService]handleMessage msg.what = BT_STOP
03-07 08:30:57.968 1188 1401 D CarTbox : [IpoManagerService]stop_bt
...
03-07 08:30:59.158 1188 1401 D CarTbox : [IpoManagerService]handleMessage msg.what = APP_STOP
03-07 08:30:59.161 1188 1401 D CarTbox : [IpoManagerService]forceStopKillPackages //AMS kill
process
03-07 08:30:59.161 1188 1401 D CarTbox : [IpoManagerService]whitelist = system
03-07 08:30:59.161 1188 1401 D CarTbox : [IpoManagerService]whitelist = com.android.bluetooth
03-07 08:30:59.161 1188 1401 D CarTbox : [IpoManagerService]whitelist = com.android.phone
03-07 08:30:59.162 1188 1401 D CarTbox : [IpoManagerService]whitelist = android.process.acore
03-07 08:30:59.162 1188 1401 D CarTbox : [IpoManagerService]whitelist = com.android.systemui
03-07 08:30:59.162 1188 1401 D CarTbox : [IpoManagerService]whitelist = com.android.car
03-07 08:30:59.162 1188 1401 D CarTbox : [IpoManagerService]whitelist = com.android.car.carlauncher
03-07 08:30:59.162 1188 1401 D CarTbox : [IpoManagerService]whitelist =
com.android.providers.media.module
...
03-07 08:31:00.459 1188 1401 D CarTbox : [IpoManagerService]lowPowerHuShutdown Done
03-07 08:31:00.460 1188 1320 D CarTbox : [IpoManagerService] Power_status_3
//IpoManagerService done

03-07 08:31:00.506 4783 4783 I IPODMAIN: Can't load library: dlopen failed: library
"/system/lib/libipod.so" not found //start ipod
```

5.2 用命令开启 IPO 时产生的关键日志

关键词：LowPowerStopIPO/ IpoManagerService

```

03-07 08:37:56.648 1188 1639 D PowerManagerService: LowPowerStopIPO
03-07 08:37:56.648 1188 1639 D CarTbox : [IpoManagerService] Power_status_6.2
03-07 08:37:56.649 1188 1639 D CarTbox : [IpoManagerService] lowPowerHuBoot start
03-07 08:37:56.649 1188 1639 D CarTbox : [IpoManagerService]huStatus:shutdown-done
03-07 08:37:56.649 1188 1639 D CarTbox : [IpoManagerService]pre requestscreen:1
03-07 08:37:56.651 1188 1401 D CarTbox : [IpoManagerService]handleMessage msg.what = AUDIO_RESTORE
03-07 08:37:56.652 1188 1401 D CarTbox : [IpoManagerService]unmuteSystem
03-07 08:37:56.653 1188 1401 D CarTbox : [IpoManagerService]handleMessage msg.what =
BOOT_ANIMATION_START
03-07 08:37:56.653 1188 1401 D CarTbox : [IpoManagerService] startBootAnimation
03-07 08:37:56.664 1188 1401 D CarTbox : [IpoManagerService]handleMessage msg.what = WAIT_MESSAGE
...
03-07 08:37:57.175 1188 1401 D CarTbox : [IpoManagerService]handleMessage msg.what = USB_RESTORE
03-07 08:37:57.175 1188 1401 D CarTbox : [IpoManagerService]start_usb

```

5.3 用电源键开启 IPO 时产生的关键日志

关键词： IPO_RESUME_REASON /LowPowerStopIPO/ IpoManagerService

```

03-07 08:45:14.497 4839 4846 I IPODMAIN: <ipo resume system>IPO_RESUME_REASON:powerkey.
03-07 08:45:14.498 4839 4846 I IPODMAIN: LowPowerStopIPO

```

5.4 因碰撞开启 IPO 时产生的关键日志

关键词： GSENSOR_COLLISION /LowPowerStopIPO

```

09-15 14:41:54.560 7545 7551 I IPODMAIN: handleEvent: event 2 mHasCallback = 0
09-15 14:41:54.560 7545 7551 I IPODMAIN: IPO_SHUTDOWN_PROP: shutdown-done
09-15 14:41:54.560 7545 7551 I IPODMAIN: sendBroadcastMessage android.intent.action.GSENSOR_COLLISION
09-15 14:41:54.560 7545 7551 I IPODMAIN: sendBroadcastMessage(): Action: c, Value: 2
09-15 14:41:54.603 7545 7551 I IPODMAIN: LowPowerStopIPO
09-15 14:41:54.603 1174 7404 D PowerManagerService: LowPowerStopIPO

```

5.5 进入 IPO 后，如何排查功耗高的原因

1. cmd 命令：dumpsys power

检查是否存在 partial WakeLock。

2. 检查 UART 日志，排查系统被谁唤醒。

3. cmd 命令：cat /sys/kernel/debug/wakeup_sources

检查 Kernel wakeup_sources。

5.6 IPO 调试命令说明

1. 默认 Kernel 开机十几秒后，UART 不会打印日志，输入以下命令可以让 UART 日志持续打印（重新开机也会生效）：`adb shell setprop persist.vendor.uartconsole.enable 1`。
2. 进 IPO 会断开 USB adb 连接，如果不想断开，可以开机后输入以下命令：
`adb shell setprop sys.ipo.usb 1`
3. 进入 IPO 命令：`adb shell service call power 56`
4. 退出 IPO 命令：`adb shell service call power 57 i32 1`

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。